

Failure (watch it loop, then measure the cost) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real safety net in code), **Advanced** (measuring the actual token cost). Read Basic first — this exercise is about *watching* something you'll otherwise only ever read about, so don't skip straight to the numbers.

- [Hint 1 — The idea, and the safety net](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the safety net

Basic Version

Never actually unlimited — even for this exercise. Temporarily raise `max_iterations` way up (e.g. `1000`) on a task designed to keep needing "just one more" tool call, and add a real wall-clock timeout on top, so a run that's technically under the step limit still can't run for more than, say, 30 seconds. Watch it print step after step. Then stop it, and put your real limit (5) back.

Intermediate Version

You need 2 separate things: a task the model can't actually resolve (an **adversarial prompt**), and a **hard stop that isn't just the step count**. For the adversarial prompt, the easiest reliable option is a tool that always returns something that looks like it needs another step — e.g., a tool that always replies "Try again with a different city name" no matter what it's given, so the model keeps retrying forever, believing progress is possible. For the hard stop, `max_iterations=1000` alone isn't actually a safety net by itself here — it bounds the *step count*, not the *time* or *cost* spent getting there, and 1000 steps of real API calls is a lot of both. Add a wall-clock check inside the loop itself.

Advanced Version

Once you've watched it loop and stopped it, the second half of this exercise is a *measurement*, not a demo: how many tokens did the adversarial run actually burn before you stopped it, versus a single direct call that needs no tool at all? The OpenAI API returns token counts on every response — `response.usage.total_tokens` — so you don't have to estimate; sum it across every step of the looping run, and compare that sum against the `total_tokens` of one plain, no-tool call. Think about *why* the ratio matters more than the raw numbers: a 50-step loop isn't "50x the cost" of 1 call, because each step also resends the entire growing message history — the cost grows faster than the step count does.

Difference between Basic, Intermediate, and Advanced: Basic says "watch it loop, safely, then measure." Intermediate gives the 2 concrete pieces that make "safely" actually true — a reliable adversarial task, and a wall-clock timeout that `max_iterations` alone doesn't provide. Advanced turns "measure" into something specific and real: pulling actual token counts from the API instead of guessing, and understanding why the cost curve is worse than linear in step count.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an adversarial tool that always says "try again" no matter what

temporarily:
    max_iterations = 1000
    also add: if more than 30 seconds have passed, stop and raise

run an adversarial task through run_agent()
watch the steps print, one after another
let the timeout stop it (don't just Ctrl+C partway through -- let the
    safety net you built actually do its job)

put max_iterations back to 5 (or whatever your real limit is)
```

Here's almost the whole thing — fill in the adversarial tool yourself:

```
# loop_safety_cost_practice.py
import time

def run_agent_with_timeout(task, max_iterations=1000, timeout_seconds=30):
    start = time.time()
    messages = [{"role": "user", "content": task}]
    for step in range(max_iterations):
        if time.time() - start > timeout_seconds:
            raise TimeoutError(f"Stopped after {timeout_seconds}s at step {step}")
        # ... same body as run_agent() from build_react_loop ...
```

Intermediate Version

```
function always_ask_again_tool(anything) -> str:
    return "That didn't work, please try a different approach and call the tool a

function run_agent_with_timeout(task, max_iterations, timeout_seconds):
    start = current time
    messages = [starting task]
```

```

for step in range(max_iterations):
    if elapsed_time > timeout_seconds:
        raise TimeoutError naming the step it stopped at
    (same body as run_agent: call model, run tool if requested, append to messages)
    raise MaxIterationsExceeded if the loop finishes without a TimeoutError first

```

```

# loop_safety_cost_practice.py
import time

def always_ask_again_tool(city: str) -> str:
    return "That didn't work, please try a different approach and call the tool again"

def run_agent_with_timeout(task: str, max_iterations: int = 1000, timeout_seconds: int = 30):
    start = time.time()
    messages: list[dict] = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        if time.time() - start > timeout_seconds:
            raise TimeoutError(f"Stopped after {timeout_seconds}s at step {step}")

        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=[adversarial_tool_schema]
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = always_ask_again_tool(call.function.arguments)
                messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
        else:
            return message.content

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps")

```

Run this, watch it print steps until the `TimeoutError` fires, then move to the Advanced Version below for the measurement part.

Advanced Version

```

looping_total_tokens = 0
run the adversarial task, catch the TimeoutError, but before that:
    for each response received in the loop, add response.usage.total_tokens

```

```

to looping_total_tokens

direct_response = one plain call, no tools, easy question
direct_total_tokens = direct_response.usage.total_tokens

print looping_total_tokens, direct_total_tokens, and the ratio between them

```

```

# loop_safety_cost_practice.py
def run_agent_with_timeout_and_token_count(task, max_iterations=1000, timeout_sec
    start = time.time()
    messages: list[dict] = [{"role": "user", "content": task}]
    total_tokens = 0

    for step in range(max_iterations):
        if time.time() - start > timeout_seconds:
            print(f"stopped at step {step}, {total_tokens} tokens used so far")
            raise TimeoutError(f"Stopped after {timeout_seconds}s at step {step}")

        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=[adversarial_tool_schem
        )
        # your turn: add response.usage.total_tokens to total_tokens,
        # then handle the tool call same as before
        ...

```

Finish tracking `total_tokens` yourself, run both the looping case and a single direct call, and compare, before checking the full [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode and near-complete code get you a safe, bounded loop you can actually watch run. Intermediate fills in the exact adversarial tool and the timeout check inside the loop body. Advanced adds real token accounting from `response.usage`, comparing the looping run's total against one direct call's — turning "agent loops are more expensive" from something you're told into something you measured yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)